10-12

10. Video Understanding

1. Video Classification

Video: 시간 순서대로 배열된 frame의 sequence. (시간 T축)핵심: 공간적 특징 + **시간에 따른 변화**를 어떻게 표현할 것인가?

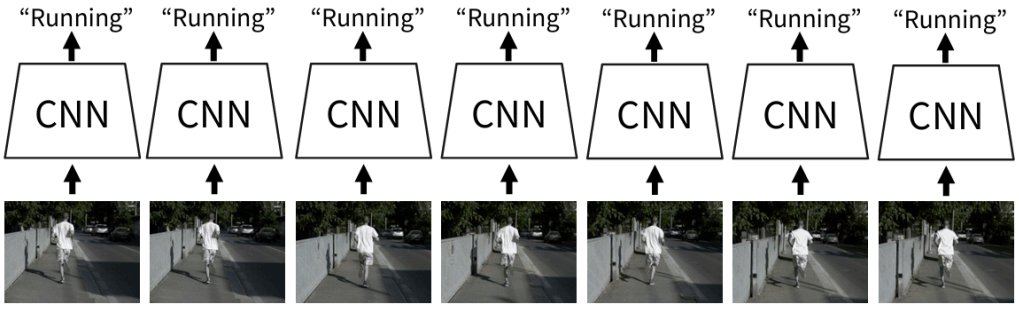
Clip 기반 학습

문제: 높은 FPS + 큰 해상도 + 긴 영상 ⇒ 메모리·연산 비용 증가.해결: **Temporal Downsampling + Spatial Downsampling → 짧은 Clip 사용.**

- Training: 원본 영상에서 **짧은 Clip**을 추출해 학습.
- Testing: 여러 Clip의 예측을 평균 → **전체 Video 예측.**

2. Frame Fusion

2.1 Single-Frame CNN

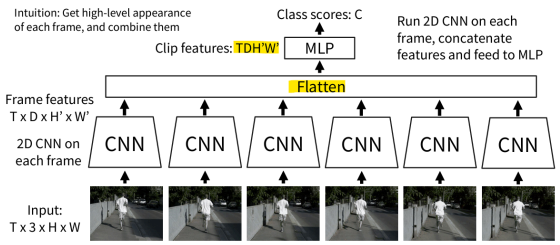


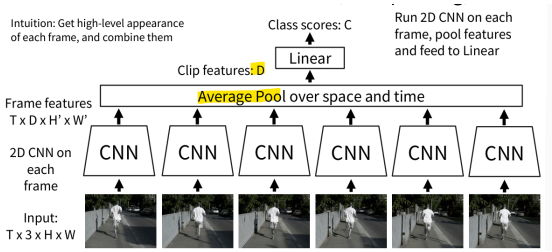
각 frame → 독립적인 2D CNN 처리 → 예측 확률 평균.

- 시간적 관계 X.
- 외형만으로도 행동을 구분할 수 있어 **강한 baseline**.
- Frame Sampling: 일정 간격 / 무작위 / 중요한 frame을 적응적으로 선택.

2.2 Late Fusion

각 frame의 **고수준 feature를 추출한 뒤 결합**.



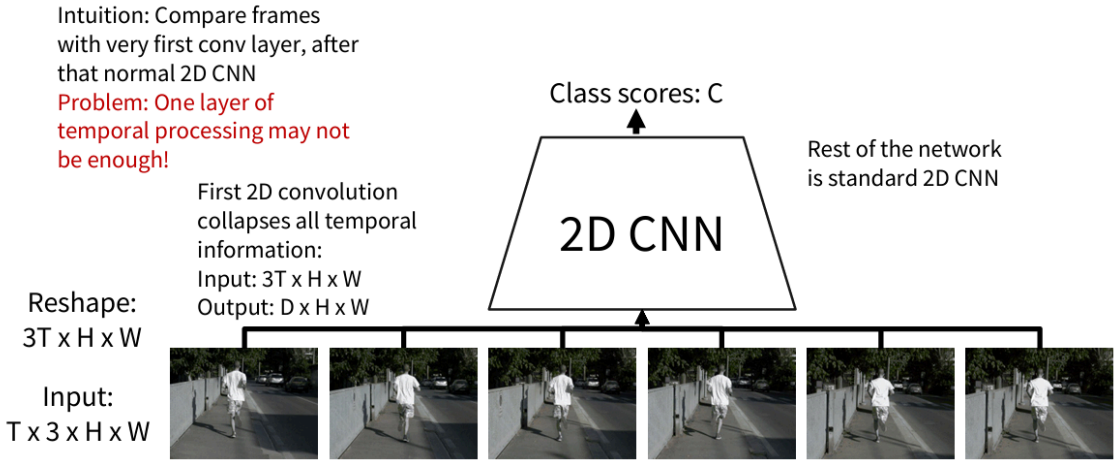


- **FC 방식**: 전체 feature를 flatten·concatenate → MLP → class score.
  - frame 수 증가 ⇒ FC layer의 parameter 수 증가.
- **Pooling 방식**: 시간·공간 pooling → D차원 vector → Linear Classifier.
  - parameter 수 감소. pooling 과정에서 세부 정보 손실 가능.

한계: frame을 독립적으로 처리 → 결합 전에 **low-level motion 정보**가 사라질 수 있음.

2.3 Early Fusion

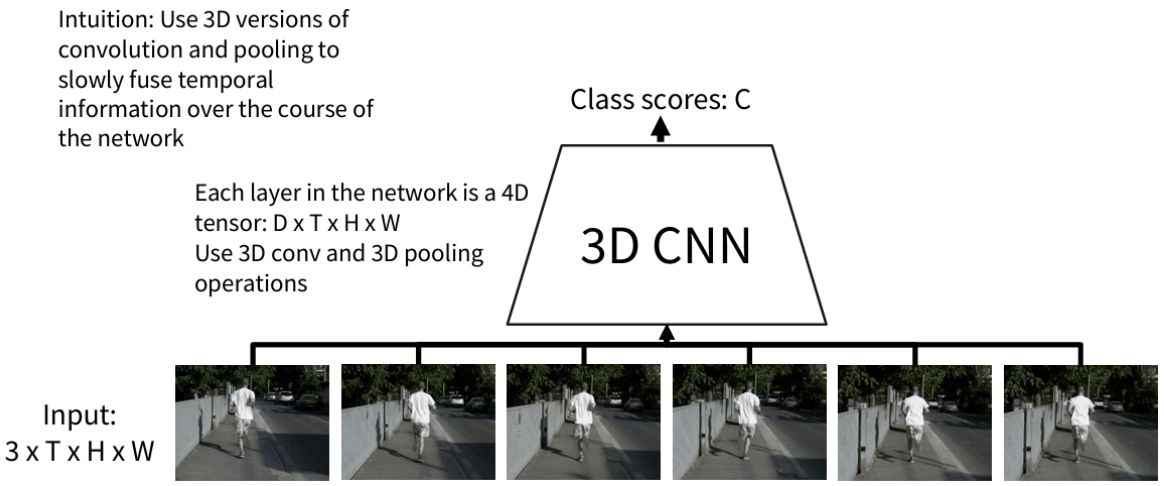
frame을 channel 방향으로 쌓아 **첫 convolution에서 시간 정보 결합**.



- 첫 convolution: 3T개 channel을 함께 처리 → frame 사이의 저수준 변화 비교.
- 이후: 별도의 시간 축 없이 일반적인 2D CNN으로 처리.

한계: 시간 정보 처리가 첫 layer에 집중됨.

2.4 Slow Fusion (3D CNN)



여러 layer에 걸쳐 **시간·공간 정보를 점진적으로 결합**.

- **시간·높이·너비의 세 축**을 따라 이동.
- 3D convolution + activation + 3D pooling을 쌓아 시공간 특징 학습.

(+) Early Fusion과의 차이

- Early Fusion: **T 정보 파괴**. → 공간 축으로만 이동. 같은 움직임도 발생 시점이 다르면 별도로 학습해야 할 수 있음.
- 3D Convolution: filter가 짧은 **시간 구간 K_t**를 포함. → 같은 움직임이 다른 시점에 나타나도 동일한 filter로 검출.

⇒ 시간 축의 parameter sharing.

(+) C3D: VGG 방식의 3D CNN.

한계: 3D convolution의 높은 연산 비용.

3. Appearance & Motion

외형이 불분명해도 움직임만으로 행동 인식 가능. ⇒ **Appearance와 Motion을 분리해 처리.**

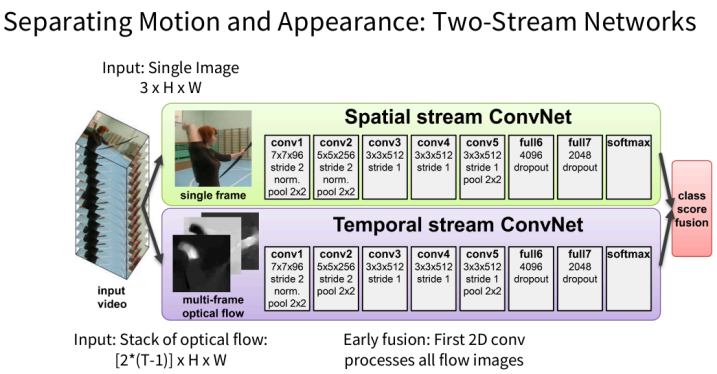
3.1 Optical Flow

연속된 두 frame 사이의 **pixel 이동량을 추정한 변위장**. $F(x, y) = (d_x, d_y)$

- d_x: 수평 이동량. d_y: 수직 이동량.
- 밝기 유지 가정 → $I_{t+1}(x + d_x, y + d_y) \approx I_t(x, y)$

결과: 위치마다 2차원 이동 vector → 국소적인 motion 표현.

3.2 Two-Stream Network



- Spatial Stream: Appearance 기반 예측. (입력: 3 × H × W)
- Temporal Stream:Motion 기반 예측. (입력: 2(T−1) × H × W)

Prediction Fusion: 두 stream의 예측 결합 → 최종 행동 분류.

(UCF-101 예시) Spatial 73.0% / Temporal 83.7% / 결합 86.9%.

⇒ motion이 강한 단서일 때 유용. appearance와 상호 보완적.

4. Long-Term Temporal Structure

문제: 짧은 Clip의 motion만으로는 **시간적으로 떨어진 사건의 관계**를 파악하기 어려움.

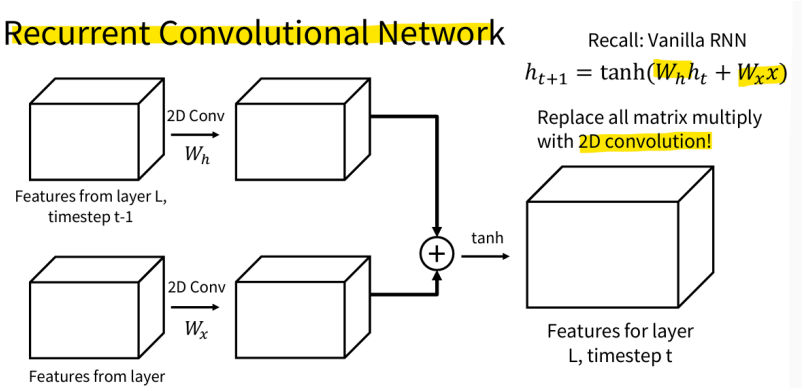
4.1 CNN + RNN

frame·Clip → CNN feature → RNN/LSTM → 예측.

- **2D/3D CNN**: frame별 시/공간 특징 추출.
- **RNN**: 이전 시점의 정보를 hidden state에 누적.

메모리 절약: 사전학습 CNN 고정시킴 + RNN만 학습 → CNN의 backward 생략.

4.2 Recurrent Convolutional Network(RCN)



RNN의 hidden state를 vector 대신 **C × H × W feature map**으로 유지.

RNN의 행렬곱 → 2D convolution으로 교체.

- 공간 구조를 유지하며 시간 정보 누적.
- layer마다 다른 weight, 시간에 따라 같은 weight 공유.
- 이론적으로 전체 과거 입력에 의존.

한계: 이전 시점 계산 필요 → 시간 축의 병렬 처리 어려움.

5. Spatio-Temporal Self-Attention

5.1 Nonlocal Block

Clip 내부의 모든 시공간 위치 사이의 관계를 직접 계산.

1. 각각의 1 × 1 × 1 Conv → Query, Key, Value 생성.
2. 시간·공간 위치를 펼치기 → N = THW개 위치.
3. Query–Key 유사도 + Softmax → Attention Weight.
4. Value 가중합 → 위치별 context feature.
5. 1 × 1 × 1 Conv로 channel 복원 → 입력과 Residual Addition.

5.2 Video Vision Transformer

- **Factorized Attention:** Spatial Attention과 Temporal Attention을 분리.
- **Multiscale Transformer:** 중간 pooling으로 token 수 감소.
- **Video Masked Autoencoder:** 일부 시공간 정보를 가리고 복원하는 사전학습.

예: TimeSformer, ViViT / MViT / VideoMAE.

6. I3D: Inflating 2D Networks to 3D

목표: 검증된 **2D CNN 구조 + 사전학습 weight**를 비디오에 재사용.

- **Architecture Inflation**: 2D convolution·pooling에 **시간 축** 추가.
  - 예: 3 × 3 Conv → 3 × 3 × 3 Conv.
- **Weight Inflation**: 2D kernel을 시간 방향으로 **K_t번 복제한 뒤 K_t로 나누기**.
  - 각 시간 slice의 weight = 기존 2D weight / K_t
  - ⇒ 동일한 frame을 반복한 입력: 시간 경계 효과가 없는 위치의 convolution 출력이 기존 2D와 같음.
- ImageNet weight로 초기화 → Video 데이터로 fine-tuning.

7. Video Model Visualization & Detection

- **Feature Visualization**: 목표 class score가 커지도록 RGB 입력과 Optical Flow에 Gradient Ascent.
- **Temporal Action Localization**: 긴 영상에서 **행동의 class + 시작·종료 시점** 예측.
  - ⇒ 시간 축의 proposal(예상 지점) 생성 → proposal별 행동 분류·구간 조정.
- **Spatio-Temporal Detection**: 시간 구간 + 행동하는 사람의 공간 위치 검출.
  - ⇒ 누가 / 어디서 / 언제 / 어떤 행동을 하는가?

8. Audio-Visual Understanding

- **Visually-Guided Audio Source Separation**: 혼합된 Audio + 영상 속 소리 source→ source별 Audio 분리.
  - 화자의 얼굴·입 움직임 → 해당 화자의 음성 추출.
  - 악기의 시각 정보 → 해당 악기 소리 분리.
- **Audio-Visual Learning**
  - 영상 feature + Audio feature → 행동 인식.
  - 이미지 patch + Spectrogram patch → Transformer 입력.
  - 일부 시각·청각 정보 masking → 복원하며 사전학습.

9. Efficient Video Understanding

문제: 긴 영상의 모든 Clip과 Modality → 비용 증가.

- **처리량 줄이기:** 모델 자체를 효율화. Clip 선택(중요한 구간만 추출. SCSampler.). Modality 선택(필요한 입력만 사용) 등.
- **Listen to Look**: 가벼운 Audio를 preview로 사용 → 중요한 Video 구간 선택.
  - 시각·청각 feature 결합 → LSTM으로 정보 누적 → 다음에 볼 구간 결정.
  - ⇒ 모든 구간을 균일하게 처리하는 대신, 중요한 순간에 연산 집중.

---

11. Large-Scale Distributed Training

1. GPU & GPU Cluster

GPU: 많은 연산을 병렬로 처리하는 processor.

- **SM(Streaming Multiprocessor)s**: GPU에서 연산을 실행.
  - 구성: 메모리 계층. FP32 Cores(scalar 연산).Tensor Cores(matrix 연산). Mixed Precision.
  - ⇒ 큰 행렬 연산을 **Tensor Core**에 효율적으로 배치하는 것이 중요.
- **FLOPs**: 부동소수점 연산 횟수. FLOP/s: 초당 처리할 수 있는 연산량.

GPU Cluster: 여러 GPU를 연결해 **하나의 큰 모델을 함께 학습**.

- Server → Rack → Pod → Cluster
- ⇒ 어떤 GPU끼리 얼마나 자주 통신하는가가 중요.

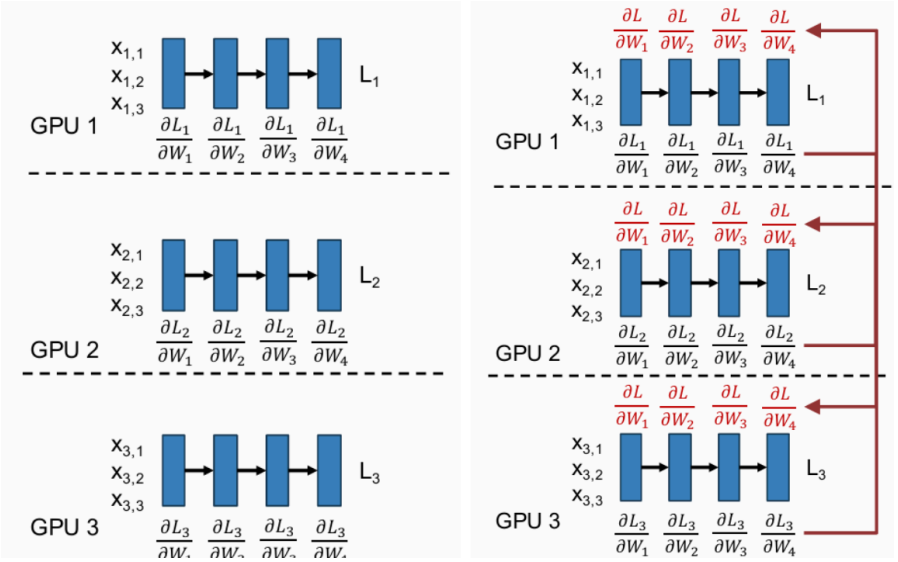
2. Data Parallelism: DP

2.1 DP(Data Parallelism)

각 GPU에 **동일한 모델을 복제 → 서로 다른 data를 처리**.각 GPU의 평균 gradient를 다시 평균 → 전체 batch의 gradient.

Global Batch Size = M × N (GPU M개, GPU당 batch size N)⇒ $g = (g_1 + g_2 + ... + g_m) / M$

학습 과정



- Forward: 각자 데이터에 대해 연산
- Backward: 각자 data에 대해 gradient 계산
- All-Reduce: 각 W_i별 평균 → GPU에게 다시 나눠줌

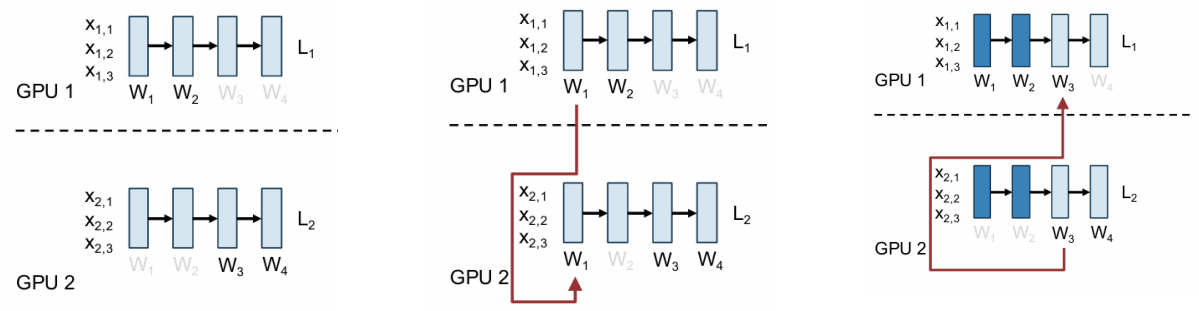
⇒ Backward + Gradient 통신을 겹쳐 실행 가능.

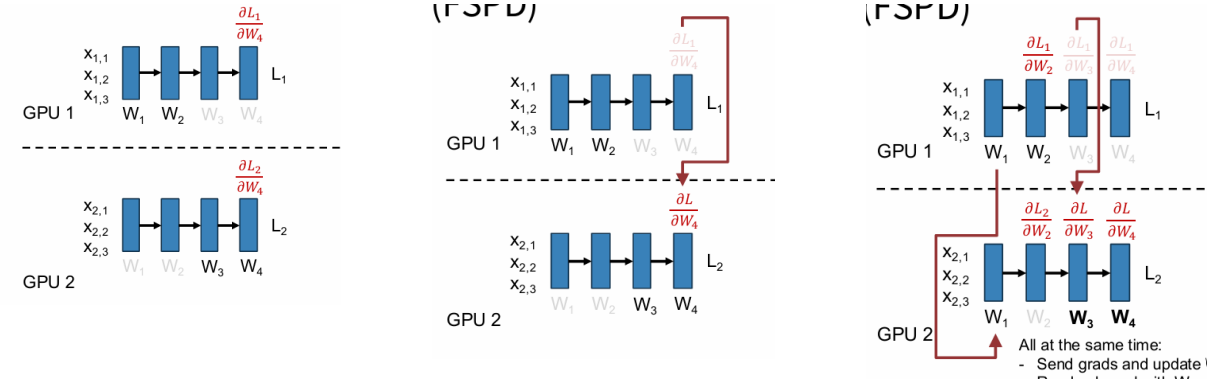
한계: 모델 메모리모든 GPU가 전체 모델을 저장 → GPU 수를 늘려도 **GPU당 모델 메모리는 감소하지 않음**.(Parameter, Gradient, Activation 등 저장)⇒ 모델 상태를 GPU에 나눠 저장하는 **Sharding**.

2.2 FSDP(Fully Sharded Data Parallelism)

1개의 model에 대해, Parameter·Gradient·Optimizer State를 GPU에 분산 저장.각 GPU는 자기 data로 전체 모델을 계산하되, 필요한 layer의 weight만 일시적으로 가져옴.

계산 방식



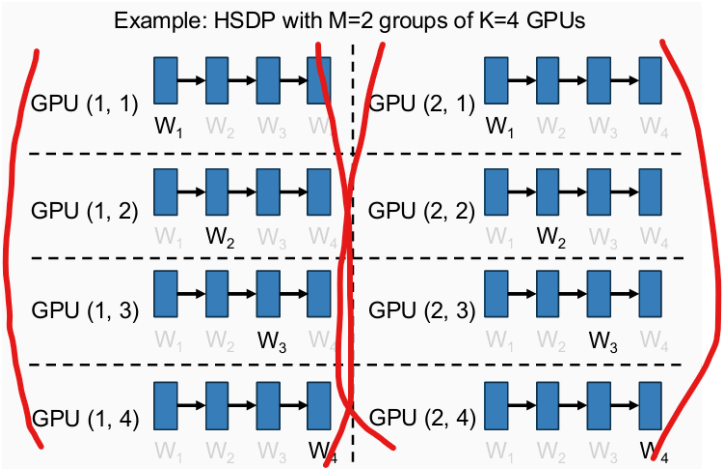


- Forward: 필요한 weight 수집 → 해당 layer 계산 → 가져온 임시 weight 삭제.
- Backward: weight 다시 수집 → local gradient 계산 → 담당 GPU로 gradient 계산 → 담당 weight 갱신.

Trade-off: 모델 메모리 감소 ↔ weight를 가져오는 통신 증가.

- 전체 weight를 계속 저장할 필요 없음.
- 현재 layer를 계산하는 동안 다음 layer의 weight를 미리 가져오기.
- Gradient 전송·갱신도 다른 layer의 backward와 겹치기.

2.3 HSDP(Hybrid Sharded Data Parallelism)



GPU를 여러 group으로 나눔. **각 group에 모델 하나를 분산 저장**하고, 전체 GPU는 서로 다른 data 처리.**- Group 내부:** FSDP → 모델 상태 분산.**- Group 외부:** DP → 대응하는 shard의 gradient 동기화.

Communication 비용 고려

- Group 내부: forward weight + backward weight + gradient 통신 → 빠른 네트워크 사용.

- Group 외부: gradient 동기화 → 상대적으로 느린 네트워크 사용.

(+) Activation Checkpointing

문제: 모델 상태를 분산해도 **Activation이 메모리를 차지**. (Backward는 각 layer의 forward activation을 필요로 함.)해결: 저장과 재계산

- **Forward+Backward(기존)**: Forward activation을 전부 저장 → backward에서 재사용.
- **Full Recomputation**: Activation을 저장하지 않고 backward 시 필요한 값을 처음부터 재계산.
- **Activation Checkpointing**: **일부 activation만 저장 → 필요한 구간만 재계산.**
  - Checkpoint 저장 → 중간 activation 해제 → backward에서 checkpoint부터 복원.
  - ⇒ 추가 연산으로 메모리를 절약. 더 큰 모델·batch를 처리.

3. GPU 활용률: HFU & MFU

- **HFU (Hardware FLOPs Utilization)**
  - GPU의 **이론적 최대 연산량** 대비 **실제로 수행한 연산량**의 비율.
  - 한계: Activation 재계산 등도 연산으로 집계 → 모델 학습에 필요한 연산과 구분하기 어려움.
- **MFU (Model FLOPs Utilization)**
  - GPU의 **이론적 최대 성능** 대비 **필수 모델 연산에 사용된 비율 (Activation 재계산 비용 고려 X)**.⇒ 학습 시스템 전체의 효율 평가 가능.
  - 병렬화 설정의 목표: MFU 최대화.

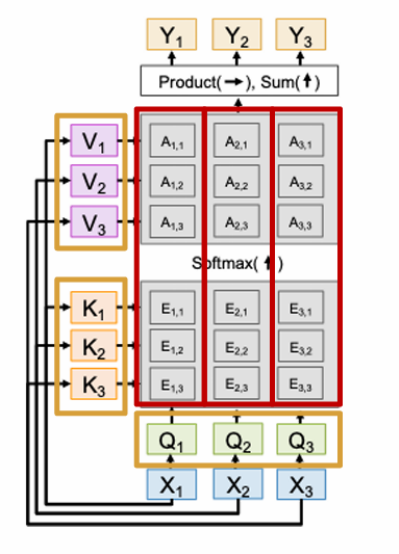
⇒ 최신 GPU: 연산 성능이 memory bandwidth보다 빠르게 증가하면 MFU가 낮아질 수 있음.

4. Context Parallelism: CP (Sequence 분배)

하나의 긴 sequence를 여러 GPU에 나눠 처리.

- Normalization·Residual: 각 token에서 독립적으로 처리 가능.
- MLP·QKV Projection: 각자의 token 처리 + weight gradient 동기화.
- **Attention:** 다른 GPU의 token도 참조해야 하므로 추가 통신 필요.

4.1 Ring Attention

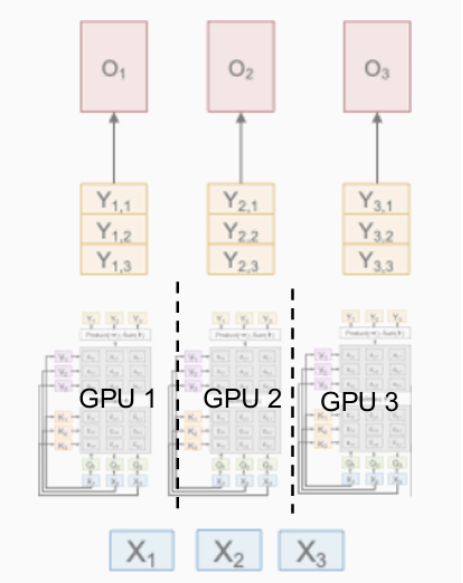


block으로 나누어 분배 → 자기 Query와 전달받은 Key·Value의 관계를 누적 계산.

Outer Block → Query는 각 GPU에서 처리.**Inner Block → Key·Value block을 GPU 사이에 순환**.

- 긴 sequence로 확장 가능.
- 통신과 Attention 계산을 겹치는 구현이 복잡함.

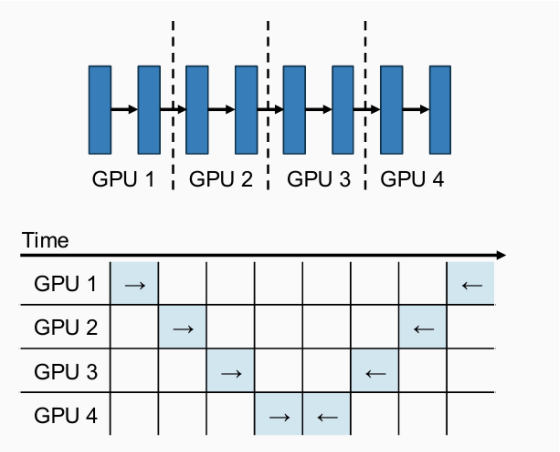
4.2 Ulysses

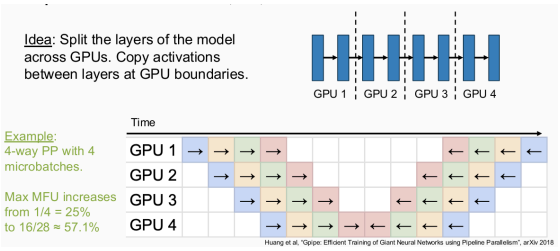


Sequence로 나눈 QKV를 재배치 → **Attention Head별로 GPU에 분배**.각 GPU가 담당 head의 전체 sequence Attention 계산 → 결과를 다시 sequence 기준으로 배치.

- Attention Matrix 자체를 분할하는 것보다 구현 단순.
- 병렬화 규모가 head 수에 제한됨.

5 Pipeline Parallelism: PP (layer 분배)



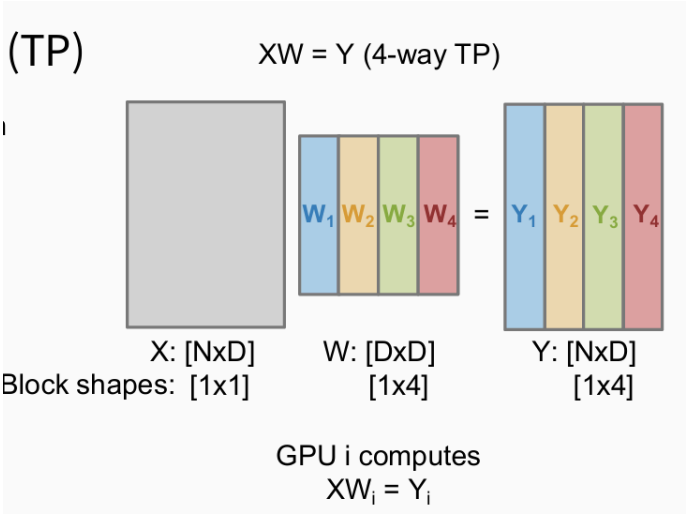


모델의 layer 단위로 여러 GPU에 나눠 배치.

문제: 앞 단계의 결과를 기다리는 동안 다른 GPU가 대기. ⇒ **Pipeline Bubble:** idle 발생.
해결: 하나의 batch를 여러 **microbatch**로 분할 → pipeline에 연속 투입.

- 서로 다른 microbatch가 여러 GPU에서 동시에 진행.
- Bubble 비율 감소 → 활용률 증가.
- 동시에 유지하는 activation 때문에 메모리 비용 증가.

6. Tensor Parallelism: TP (Dim 분할)



행렬 연산 → 분할 하나의 layer 내부 weight matrix를 나눠 계산.⇒ GPU마다 출력 feature의 일부 생성.

- FSDP: weight를 가져와 각 GPU가 해당 layer 전체 계산.
- TP: 각 GPU가 weight 일부로 해당 layer의 일부 계산.

6.1 연속된 두 Layer

문제: layer마다 전체 출력을 모으면 통신 대기 발생.해결: **첫 weight는 열 방향, 다음 weight는 행 방향으로 맞춰 분할.**

⇒ 통신을 줄일 수 있지만, 여전히 layer 내부의 잦은 동기화 필요 → 빠른 GPU 연결이 중요.

7. ND(N-Dimensional) Parallelism

큰 모델에서는 **DP + CP + PP + TP를 동시에 사용**.

- 전체 GPU 수 = DP × CP × PP × TP
- 실무 사용
  - 모델 상태가 큼 → FSDP·HSDP.
  - Activation이 큼 → Checkpointing.
  - Sequence가 김 → CP.
  - Layer를 나눠야 함 → PP.
  - Layer 내부 연산을 나눠야 함 → TP.

GPU 수만 늘리면 통신·대기 비용도 증가.

⇒ **메모리 한도와 통신 구조를 고려해 조합하고, MFU로 평가.**

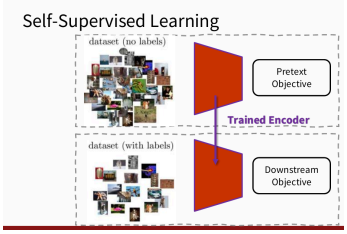
---

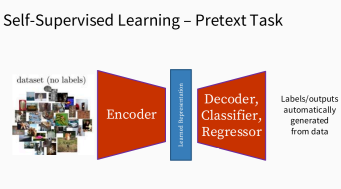
12. Self-Supervised Learning

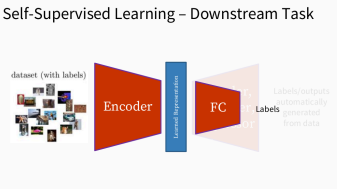
1. Self-Supervised Learning

Self-Supervised Learning: data 자체에서 학습 목표를 만듦 → **label 없이 representation 학습**.

1.1 Pretext Task & Downstream Task







Pretext Task: 대량의 Unlabeled data → Encoder 학습. 자동 생성한 정답을 target과 비교.(예: 회전 각도 예측, 가려진 영역 복원.)

Downstream Task: 사전학습된 Encoder + 새로운 출력 모듈 → **실제 labeled data**로 **task**(분류·탐지·segmentation) **진행**.

- Pretext Task의 출력 모듈은 제거.
- 학습한 feature를 실제 문제에 전이.

⇒ Pretext Task로 특징을 학습하고, **Downstream에서 그 특징을 실제 task에 활용하도록 만듦**

1.2 평가 방식

- **Linear Probing:** Encoder 고정 + Linear Classifier만 학습 → 기존 표현의 품질 평가.
- **Fine-Tuning:** Encoder까지 추가 학습 → 실제 과제에 적응.
- Clustering/t-SNE: feature의 군집/분리 양상 확인.
- generalization/robustness/Computational Efficiency 함께 평가.

2. Image Transformation 기반 Pretext Task

원본 이미지 + 변환 적용 → **변환 정보 / 원본을 정답으로 사용**.

- **Rotation Prediction:** 이미지를 0° / 90° / 180° / 270° 회전 → 적용된 각도 예측.
- **Relative Patch Location:** 중앙 patch + 주변 patch 입력 → 주변 patch의 상대 위치 예측.
- **Jigsaw Puzzle:** 이미지를 3 × 3 patch로 분할 → 순서 섞기 → 올바른 배열 예측.
- **Image Inpainting:** 일부 영역을 가리기 → Encoder → Decoder → **가려진 pixel 복원**.
  - $L = L_{recon} + L_{adv}$
  - Reconstruction Loss: 가려진 영역의 복원 오차.

    - Adversarial Loss: 실제 이미지처럼 자연스러운 결과 유도.
- **Colorization**: LAB 색 공간에서 **밝기 L → 색상 a, b 예측** (물체·장면과 색의 관계 학습.)
    - Split-Brain Autoencoder: 입력 channel을 두 집합으로 나눠 **서로 예측**.→ channel 사이의 예측에도 적용 가능.

3. Video의 Pretext Task

3.1 Video Colorization

Idea: 같은 물체의 색이 시간에 따라 유지되도록 학습 → **frame 사이의 대응 관계** 학습.

Color Propagation: 색상 있는 Reference Frame + 흑백 Target Frame → Reference의 색을 Target으로 전달.목표: 같은 물체가 어디로 이동했는지 파악 (traking)

1. 두 frame의 위치별 feature 추출.
2. Feature 유사도 → Reference 위치에 대한 Attention 계산.
3. Reference의 색을 Attention Weight로 가중합.
4. Target의 실제 색과 비교해 학습.

⇒ 색을 새로 생성하는 대신, **어느 위치의 색을 가져올지** 학습.

Downstream Task: 학습한 Attention을 이용해 색 대신 다른 정보 전달.

- Segmentation Mask → object 영역 추적.
- Pose Keypoint → 관절 위치(Keypoint) 추적.

⇒ Colorization이라는 Pretext Task에서 **pixel correspondence** 획득.

4. Masked Autoencoder: MAE

MAE: 이미지의 많은 patch를 가리고, **보이는 일부만으로 원본 복원**.

Encoder 과정

1. 이미지 → patch로 분할. 무작위로 (약 75%) masking. 2. **남은 원본(25%)** + Positional Embedding → ViT Encoder.3. Transformer Block들을 통해 **feature extraction**

Decoder 과정

1. mask된 위치에 **학습 가능한 (동일) Mask Token** 삽입2. Positional Embedding 추가 3. 전체 토큰 시퀀스 → 작은 Transformer Decoder → **pixel 예측**. (이미지 재구성)

- **큰 Encoder + 작은 Decoder:** 비대칭 Autoencoder.
- **Decoder**는 복원에만 사용 → 사전학습 후 제거.

Reconstruction Loss

Masking된 patch에서만 MSE 계산. 보이는 patch는 loss에서 제외.⇒ 제한된 정보로 이미지 구조를 추론하도록 학습.

Linear Probing, Full Fine-tuning

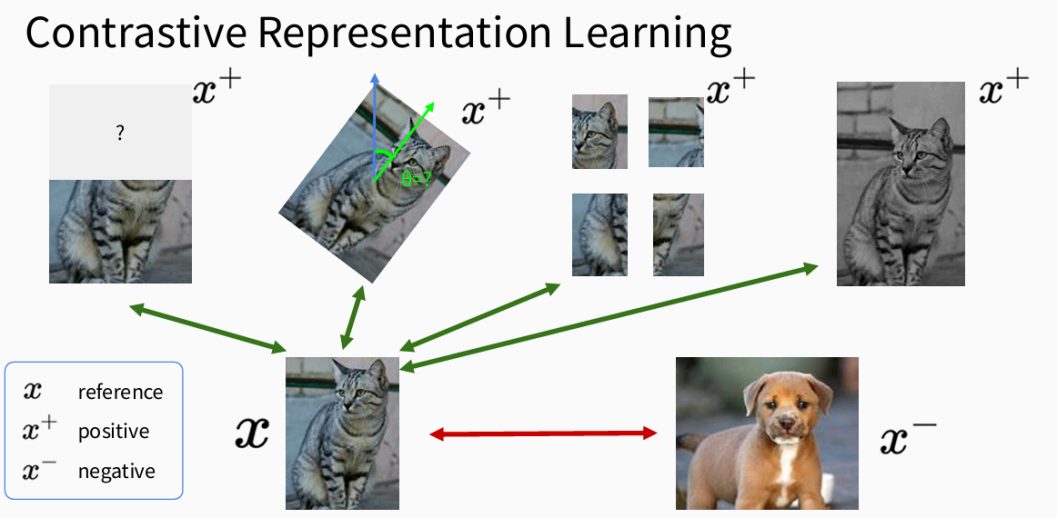
- Linear Probing: Encoder이 얼마나 좋은지 평가하기 위함
- Full Fine-tuning: Encoder을 task에 맞추기 위해 조정

5. Contrastive Representation Learning

문제: **Pretext Task를 개별적으로 설계**해야 함. feature이 특정 Pretext task에 종속될 수 있음.해결: 특정 task를 해결하는 대신, **sample 사이의 관계** 학습⇒ 다양한 downstream task에 활용할 수 있는 일반적인 특징 추출

5.1 기본 Idea

같은 원본 이미지를 기준으로 변형 구성.



- **Reference / Anchor (x):** 기준 sample.
- **Positive (x+): 같은 이미지**에서 생성한 다른 view.
- **Negative (x-): 다른 이미지**에서 가져온 sample.

학습 목표: Attract(Positive feature는 가깝게). Repel(Negative feature는 멀게).

5.2 Loss Function

Positive 1개 + Negative N−1개 → **N개 후보 중 Positive 찾기**.

$$L = -\mathbb{E}_X \left[\log \frac{\exp(s(f(x), f(x^+))}{\exp(s(f(x), f(x^+)) + \sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-))} \right]$$

score for the positive pair / score for the N-1 negative pairs

- s^+: Anchor–Positive 유사도.
- s^-: Anchor–Negative 유사도.

$$MI[f(x), f(x^+)] - \log(N) \geq -L$$

MI(Mutual Information): 두 x, x+ 사이에 얼마나 많은 정보가 공유되는가?

⇒ (Loss 최소화 방향) = (Positive 표현 사이의 **MI 하한을 높이는 방향**.)= 많은 Negative를 확보하는 것이 주요 설계 요소.

5.3 Encoder 학습

같은 이미지에서 변형된 두 이미지의 특징은 비슷하게, 서로 다른 이미지의 특징은 다르게 학습

⇒ label 없이 유용한 이미지 특징 추출하는 Encoder 학습

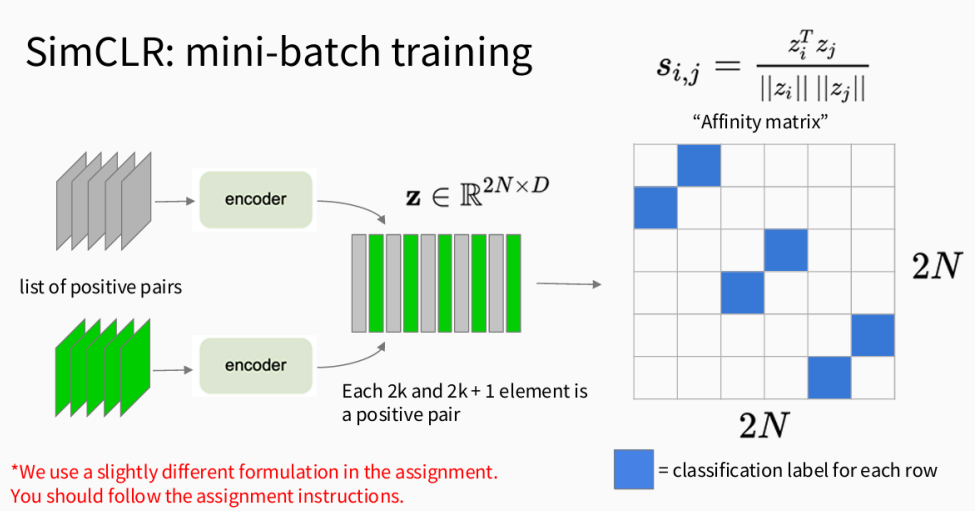
(1) SimCLR

SimCLR 구조

1. 원본 이미지 x에서, 서로 다른 **Data Augmentation** 적용 (예시: 2회) (→ Positive Pair.)2. 두 이미지 → **Encoder** 통과시켜 특징 벡터 h 추출3. **Projection head g** 통과 → Contrastive Learning에 사용할 **특징 z** 얻기4. Contrastive Loss → 학습 진행.(유사도: Cosine Similarity.) (Temperature τ로 유사도를 조절한 뒤 Softmax 적용.)5. 이후 Projection Head는 제거. Encoder 출력 h를 Downstream task에 활용.

SimCLR training

1. **mini-batch** 구성: 원본 이미지 N개 → (예시: 2개 변형) 2N개 이미지 생성.2. 동일한 원본 이미지에 생성된 두 변형 → Positive, 그외 → Negative 설정.3. Positive Pair끼리 유사도를 높이고, 나머지 pair의 유사도를 낮추는 Loss 계싼.4. Reference 교체, 반복.



Affinity Matrix: 전체 anchor 사이의 유사도(Loss) 한번에 계산 계산 → 2N × 2N 행렬.

- 대각선: 자기 자신 → 제외.
- 각 행의 정답: Positive가 있는 열.

Projection Head의 역할

Contrastive Objective는 Augmentation에 대한 불변성 표현을 학습하도록 유도.그러나 학습 과정에서 유용한 정보까지 제거될 수 있음.⇒ z에서 Contrastive Loss를 적용 → **h에 더 많은 정보를 보존하도록 유도**.

(실험) Nonlinear이 Linear보다 Projection Head가 효과적.

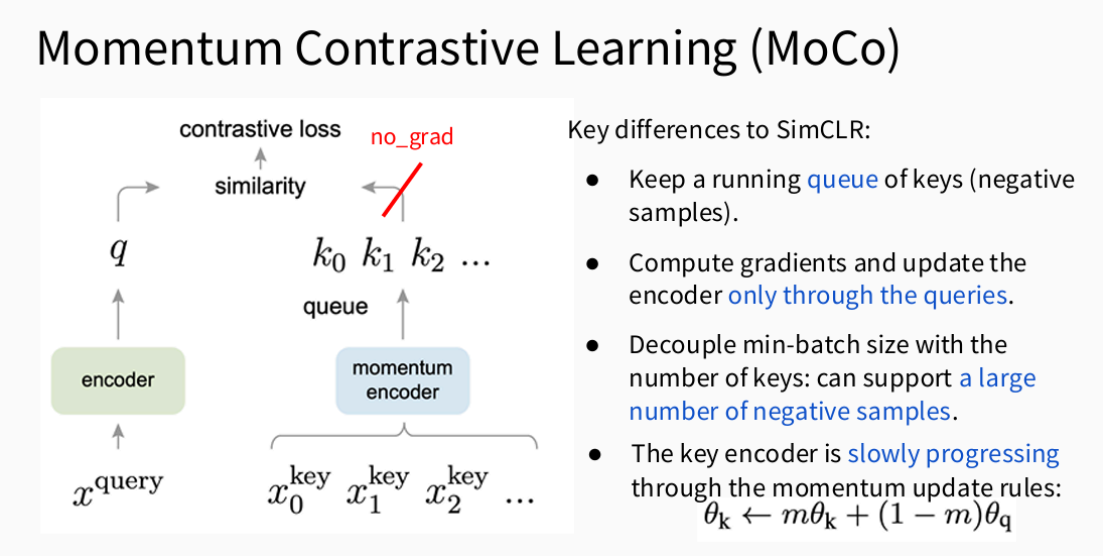
Large Batch의 한계

Batch 증가 → Negative 증가 → 표현 학습에 유리.그러나 모든 view의 forward·backward 필요 → **메모리 부담 증가**.

⇒ Batch를 크게 만들지 않고 Negative를 확보하는 **MoCo**.

(2) Momentum Contrast: MoCo

MoCo: 이전 Mini-Batch에서 추출한 특징 벡터를 Queue에 저장 + Momentum Encoder→ 많은 Negative Samples를 사용해 Contrastive Learning을 수행. 안정적 관리.



기본 과정

1. 동일한 원본 이미지 x에서 두 변형 생성 → 하나(x)는 Query, 하나(x0)는 Key Encoder에 입력.(나머지(x1, x2, ...): Negative Key. Queue에 저장.)2. 두 Enxoder → 특징 벡터 q, k0생성.3. Queue에 저장된 Negative Keys → InfoNCE Loss 계산4. **Query Encoder 업데이트**: Backpropagation 실행. (Key Encoder: Gradient로 업데이트 X.)5. **Key Encoder 업데이트**: Query Encoder의 파라미터를 반영하여 업데이트(Momentum Update: $\theta_k \leftarrow m\theta_k + (1-m)\theta_q$) **⇒ Key Encoder를 천천히 변화시켜 특징 벡터를 안정적으로 비교**.6. 새로운 Key → Queue에 추가. 오래된 Key → 제거. ⇒ 작은 batch에서도 많은 Negative를 사용 가능.

MoCo v2

MoCo 구조 + SimCLR 설계 결합.

- MoCo: Momentum Encoder + Negative Queue.
- SimCLR: Nonlinear Projection Head + Strong Augmentation.

⇒ 효과적인 Contrastive Learning 수행. Memory 부담 감소.